Deep Learning Projects

Data Import → EDA → Preprocessing → Model-building → Evaluation → Saving

---

Rice Leaf Disease Classification — Deep Learning (CNN + Transfer Learning)

4.1 Objective

Classify rice leaf images into disease categories — Bacterial Leaf Blight, Brown Spot, or Leaf Smut — a 3-class image classification problem on a very small dataset.

4.2 Data Import

path = kagglehub.dataset_download("vbookshelf/rice-leaf-diseases")

Images live under a rice_leaf_diseases sub-folder with one folder per disease class. A DataFrame of (image_path, label) is built and shuffled, exactly as in the digit-classification project.

4.3 Exploratory Data Analysis

- 5 sample images per class displayed in a grid.
- Per-class image counts printed — the dataset is very small overall (about 120 images across 3 classes).
- Native image size checked on a sample file.

4.4 Preprocessing & Split

- 70/15/15 split → 84 training, 18 validation, 18 test images — a very small split, especially for validation and test.

```
data_generator = ImageDataGenerator(rescale=1/255, rotation_range=20,
  width_shift_range=0.2, height_shift_range=0.2, shear_range=0.15,
  zoom_range=0.2, horizontal_flip=True, fill_mode='nearest')
val_datagen = ImageDataGenerator(rescale=1./255)  # defined, but not actually used
```

- Images resized to 224×224×3 (matching MobileNetV2's expected input size).

4.5 Model 1 — Custom CNN Baseline

A 4-block CNN (Conv2D + BatchNorm + MaxPooling, filters 32→64→128→256, with dropout in the later blocks) followed by a Dense(128) classifier head, trained for 20 epochs with Adam (lr=0.0001). This served as a baseline before moving to transfer learning.

4.6 Model 2 — Transfer Learning with MobileNetV2

Given the very small dataset and the overfitting visible in the baseline CNN's train/val curves, the notebook switches to transfer learning:

```
base_model = MobileNetV2(weights='imagenet', include_top=False,
                         input_shape=(224,224,3))
base_model.trainable = False

x = GlobalAveragePooling2D()(base_model.output)
x = Dense(128, activation='relu')(x)
```

x = Dropout(0.5)(x)
predictions = Dense(3, activation='softmax')(x)
transfer_model = Model(inputs=base_model.input, outputs=predictions)

Compiled with Adam(lr=0.001), trained 20 epochs on the same generators.

4.7 Evaluation

Model	Test Accuracy	Test Loss
Custom CNN (baseline)	not explicitly reported on test set	—
MobileNetV2 Transfer Model	88.89%	0.2047

- Training/validation accuracy and loss curves plotted for both models.
- A held-out image is classified with the final transfer model: predicted 'Brown spot' at 99.37% confidence.
- A lookup table mapping class index → disease name is generated for interpreting predictions.

4.8 Model Saving

transfer_model.save("Rice_leaf_transfer_model.h5") <-- commented out

This line is commented out, so the trained model was never actually persisted to disk in this run — only a deprecation warning about the legacy .h5 format appears in the output, with no file written.

4.9 Pros

- Correctly recognizes that the dataset is far too small (~120 images) to train a deep CNN from scratch, and pivots to transfer learning with a pretrained MobileNetV2 backbone — the right strategy for small image datasets.
- Runs both a from-scratch baseline and the transfer-learning model, so the improvement from transfer learning is directly visible rather than assumed.
- Achieves a strong 88.9% test accuracy despite the tiny dataset, and adds a confidence score to individual predictions, which is useful for a real deployment (e.g. flag low-confidence predictions for manual review).
- Provides a clean class-index-to-label mapping table, avoiding confusion when interpreting predictions.

4.10 Cons, Challenges & How to Handle Them

- **Tiny validation/test sets:** 18 validation and 18 test images means each single image is worth ~5.5% of the reported accuracy — the 88.9% figure has very wide uncertainty and could shift substantially on a different random split. Fix: use k-fold cross-validation across the whole dataset (not just one 70/15/15 split), and/or collect more images per class before trusting the number for deployment decisions.
- **val_datagen defined but unused:** a non-augmenting ImageDataGenerator(rescale=1./255) is created for validation, but validation_generator and test_generator are actually built from the augmented data_generator — the same augmentation-leakage issue seen in the digit-

classification notebook. Fix: pass val_datagen (not data_generator) into flow_from_dataframe() for validation_generator and test_generator.

- **No fine-tuning of the backbone:** MobileNetV2's layers are fully frozen and never unfrozen for fine-tuning, leaving potential accuracy on the table — especially valuable once more data is collected. Fix: after the head converges, unfreeze the top N layers of MobileNetV2 and continue training briefly with a very low learning rate (e.g. 1e-5).
- **Model not actually saved:** the .save() call is commented out, so this notebook run produces no reusable model artifact — anyone wanting to deploy or hand off the model would have to retrain it. Fix: uncomment and update to the recommended format: transfer_model.save('rice_leaf_model.keras'), and add a reload + re-predict cell to confirm it works.
- **Class imbalance not explicitly checked:** unlike the digit-classification notebook, per-class image counts aren't printed here, so it's unclear whether the 3 disease classes are balanced. With only ~120 images total, even a small imbalance could bias the model. Fix: print value_counts() on the label column before splitting, and consider class_weight in model.fit() if imbalance is found.
- **No formal side-by-side comparison table:** the baseline CNN and the transfer model are evaluated with separate plots rather than a shared metrics table, making it harder to quantify the improvement transfer learning provided. Fix: evaluate both models on the identical test set and report their accuracy/loss/F1 in one table (as summarized in 4.7 above).